

SORTOWANIE PRZEZ SCALANIE

Mamy n liczb $a_1 a_2 a_3 \dots a_n$

Zadanie polega na uporządkowaniu ich od najmniejszej do największej

$$c_1 \leq c_2 \leq c_3 \leq \dots \leq c_n$$

Proste algorytmy mają złożoność
(czas działania) $O(n^2)$

Wybkie algorytmy mają złożoność
 $O(n \log n)$

Tę złożoność ma też sortowanie przez
scalanie

Scalanie :

Dane 2 cięgi (posortowane)

$$a_1 \leq a_2 \leq \dots \leq a_k$$

$$b_1 \leq b_2 \leq \dots \leq b_l$$

Zadanie polega na scaleniu ich w jeden posortowany ciąg

$$c_1 \leq c_2 \leq c_3 \leq \dots \leq c_{k+l}$$

$$Czas scalenia \approx C \cdot (k+l) = O(k+l)$$

$$\max \# \text{ porównań : } k+l-1$$

Sortowanie przez scalanie

Sortujemy ciąg $a_1 a_2 \dots a_n$

$\text{SORTUVJ}(i, j)$ {zwraca posort. ci. $a_i \dots a_j$ }

jeśli $i = j$ to zwróć a_i

$$m \leftarrow \left\lfloor \frac{i+j-1}{2} \right\rfloor$$

zwróć $\text{SCAL}(\text{SORTUVJ}(i, m), \text{SORTUVJ}(m+1, j))$

Procedurę tę wykorzystujemy na posz:

$\text{SORTUVJ}(1, n)$

$T(n)$ - czas działania dla danego rozr. n

$$n = 2^k$$

$$T(n) = 2T(n/2) + cn$$

$$T(n) = cn + 2T(n/2)$$

$$k = \log n$$

$$= cn + 2cn/2 + 4T(n/4)$$

- - -

$$= \underbrace{cn + cn + cn + \dots + cn}_k + 2^k T(n/2^k)$$

$$= cn \log n + n \cdot d = O(n \log n)$$

$f(n)$ — liczba osób na danych
 nożu n (n niekoniecznie jest
 wielokrotnością 2)

$$f(n) = f(\lfloor \frac{n}{2} \rfloor) + f(\lceil \frac{n}{2} \rceil) + n - 1$$

МНОЖЕНИЕ ДВУХ ЛИБО
СЛУЖИТЕЛЬНЫХ (n-цифровых)

Множение письменное - сложность $O(n^2)$

Алгоритм Каратсуби $O(n^{\log_2 3})$
 $\approx O(n^{1.58})$

Истинно лучшие алгоритмы о сложности
ведет к $O(n \log n)$

A - число n цифровое

$$\underbrace{a_{n-1} \dots a_{\frac{n}{2}}}_{A_1} \underbrace{a_{\frac{n}{2}-1} \dots a_2 a_1 a_0}_{A_0}$$

$$A = 2^{\frac{n}{2}} A_1 + A_0$$

- Algoritmus Karatsuby
- množina čísel n -cifrových A a B
- Teze: A a B se rozdělí na poloviny

1. Podělíme čísla na poloviny

$$A = 2^{\frac{n}{2}} A_1 + A_0 \quad B = 2^{\frac{n}{2}} B_1 + B_0$$

2. Vypočítáme alg. rekurenci pro M

$$M_0 = A_0 B_0 \quad M_1 = A_1 B_1$$

$$M_2 = (A_0 + A_1)(B_0 + B_1)$$

3. Obecný výsledek

$$AB \leftarrow 2^n M_1 + 2^{\frac{n}{2}} (M_2 - M_0 - M_1) + M_0$$

$$\begin{aligned}
 A \cdot B &= (2^{n/2} A_1 + A_0) (2^{n/2} B_1 + B_0) \\
 &= 2^n \boxed{A_1 B_1} + \boxed{2^{n/2} (A_1 B_0 + A_0 B_1)} + \boxed{A_0 B_0}
 \end{aligned}$$

M_1 M_2 M_0

$$M_2 = (A_1 + A_0)(B_1 + B_0) = A_1 B_1 + \boxed{A_1 B_0 + A_0 B_1} + A_0 B_0$$

$$\boxed{A_1 B_0 + A_0 B_1 = M_2 - M_0 - M_1}$$

2 to 3 recursive algorithm Karatsuba

$T(n)$ - czas działania dla danych n

$$T(n) = 3T(n/2) + cn$$

$$T(n) = cn + 3T(n/2) - \quad k = \log n$$

$$= cn + 3cn/2 + 9T(n/4)$$

$$= cn + \frac{3}{2}cn + \frac{9}{4}cn + \dots + \frac{3^{k-1}}{2^{k-1}}cn + 3^k T\left(\frac{n}{2^k}\right)$$

$$= \frac{3^k}{2^k} c \left(\frac{2}{3} + \frac{2^2}{3^2} + \frac{2^3}{3^3} + \dots \right) + 3^k c$$

$\sqrt[2]{\frac{1}{1-\frac{2}{3}}} = 2$

$$\leq 2c3^k + d3^k = O(3^k) = O(2^{\log 3 \log n}) = O(n^{\log 3})$$

$= O(n^{1.58\dots})$

Algorytm Euklidesa

$$\text{NWD}(a, b) = \max \{d \in \mathbb{Z} : d \mid a \text{ i } d \mid b\}$$

$$\text{NWW}(a, b) = \min \{c \in [\text{NWD}\{0\} : a/c \text{ i } b/c\}$$

$$\text{NWD}(a, 0) = \text{NWD}(0, a) = a$$

$$\text{NWD}(a, b) = \text{NWD}(a - b, b) = \text{NWD}(a - cb, b)$$

$$\text{NWD}(a, b)$$

$$\text{dopóki } b > 0$$

$$(a, b) \leftarrow (b, a \bmod b)$$

zwróć a



$$a, b \in \mathbb{N}$$

$$a \text{ DNB } b = q = \left\lfloor \frac{a}{b} \right\rfloor$$

$$a - q \cdot b = a - \left\lfloor \frac{a}{b} \right\rfloor b = a \bmod b$$

Nierówność pyth

— Para (a, b) ma cęty nas
tę samą wartość NWD

$$\text{NWD}(a, b) = \text{NWD}(b, a \bmod b)$$

— Na końcu mamy $b = 0$

$$\text{co daje } \text{NWD}(a, b) = \text{NWD}(a, 0) = a$$

Algorytm ten zawsze się kończy

bo para (a, b) jest coraz mniejsza

Pokazujemy, że ten algorytm jest efektywny.

W tym celu oszacujemy liczbę iteracji wykonanych przez algorytm.

Fakt: Po raz pierwszy od drugiej iteracji $a > b$
(bo zawsze $a \bmod b < b$)

Lema Lemat: Zauważ, że w i -tej iteracji ($i > 1$)

Fibonacci $a < F_{k+1}$ lub $b < F_k$. Wtedy para (a', b') nast po tej iteracji spełnia $a' < F_k$ lub $b' < F_{k-1}$

Důvod:

Předpoklad 1: $b < F_k \Rightarrow a' = b < F_k$

Předpoklad 2: $b \geq F_k \Rightarrow a < F_{k+1}$

$$\Rightarrow b' = a \bmod b \leq a - b < F_{k+1} - F_k = F_{k-1}$$

$a > b, b_0 \geq 1$

Uvědomte: Pokud na počátku
 $a, b < F_n$, to algoritmus Euklidese
vykoná co nejrychleji n iterací

po iteraci 1 $a < F_{n+1} \vee b < F_n$

po iteraci 2 $a < F_n \vee b < F_{n-1}$

po iteraci n $a < F_2 \vee b < F_1$ $a > b$
 $\Rightarrow b = 0$

$$F_n = \left\lfloor \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n + \frac{1}{2} \right\rfloor$$

a, b — найбольш $\Rightarrow a, b < 2^n$

$$\Rightarrow a, b < \left(\frac{1+\sqrt{5}}{2} \right)^n / \log \left(\frac{1+\sqrt{5}}{2} \right) \approx \left(\frac{1+\sqrt{5}}{2} \right)^{1.44n}$$

$$\Rightarrow a, b < F_{\lceil 1.44n \rceil}$$

$\Uparrow$
max # ітерацій alg Еукліда

Rozszerzony algorytm Euklidesa
wylicza nie tylko $\text{NWD}(a, b)$
ale także dodatkowe

$x, y \in \mathbb{Z}$, takie że $xa + yb = \text{NWD}(a, b)$

Przykład

$$a = 245 \quad b = 168$$

$$\boxed{11} \cdot 245 - \boxed{16} \cdot 168 = 7 = \text{NWD}(245, 168)$$

$$x = 11$$

$$y = -16$$

iter	a	b
1	245	168
2	168	$245 - 168 = 77$
3	77	$168 - 2 \cdot 77 = 14$
4	14	$77 - 5 \cdot 14 = 7$
5	7 NWD	$14 - 2 \cdot 7 = 0$

$$77 - 5 \cdot 14 = 77 - 5 \cdot (168 - 2 \cdot 77) =$$

$$= -5 \cdot 168 + 11 \cdot 77 = 7$$

$$-5 \cdot 168 + 11 \cdot 77 = -5 \cdot 168 + 11(245 - 168)$$

$$= 11 \cdot 245 - 16 \cdot 168$$

Obličanje odvrtnosti v $\mathbb{Z}_n$

$a \in \mathbb{Z}_n$ $a^{-1} \bmod \mathbb{Z}_n$ to take a'

$$a \cdot_n a' = (a \cdot a') \bmod n = 1$$

$$\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$$

$$a +_n b = (a + b) \bmod n$$

$$x \bmod n = x - \left\lfloor \frac{x}{n} \right\rfloor \cdot n - \text{reszta z}$$

dielecia
x przez n

$$a \cdot_n b = (a \cdot b) \bmod n$$

$\mathbb{Z}_n$ jest grupą $+$

$$(a +_n b) +_n c = a +_n (b +_n c)$$

$$a +_n b +_n c = (a + b + c) \bmod n$$

el. neutral to 0

el. odwrotny to $\begin{cases} 0 & \text{dla } a = 0 \\ n-a & \text{dla } a \neq 0 \end{cases}$

$$a +_n (n-a) = n \bmod n = 0$$

$$(a +_n b) \cdot_n c = a \cdot_n c +_n b \cdot_n c$$

$$(a \cdot_n b) \cdot_n c = a \cdot_n (b \cdot_n c)$$

1 jest el. neutral mnożenia

$$n = 6$$

2 nie ma el odwrotnej

$$2 \cdot_6 1 = 2$$

$$2 \cdot_6 2 = 4$$

$$2 \cdot_6 3 = 0$$

$$2 \cdot_6 4 = 2$$

$$2 \cdot_6 5 = 4$$

żadnego $a \neq 1$

$$5^{-1} \bmod 6 = 5, \text{ bo } 5 \cdot_6 5 = 1$$

FAKT Jeżeli $\text{NWD}(a, n) > 1$ to
 $a^{-1} \bmod n$ nie istnieje

$$\text{NWD}(a, n) = d > 1$$

$$\begin{aligned} a &= ad \\ n &= vd \end{aligned}$$

$$a \cdot_n x = a \cdot d \cdot x \text{ MOD } n = ax - ydq$$

$$\geq d(- -)$$

$$d \nmid 1$$

$$d \mid a \cdot_n x$$

$$\Downarrow$$

$$a \cdot_n x \neq 1$$

Pokażemy jak policzyć $a^{-1} \text{ mod } n$
 gdy $\text{NWD}(a, n) = 1$ ($a \perp n$ - a względnie
 pierwsze z n)

- Stosujemy rozszerzony alg Euklidesa
 dla a i n

Uy liany $x, y : xa + yn = \text{NWD}(a, n) = 1$

$$xa + yn = 1 \Rightarrow xa \equiv 1 \pmod{n}$$

$$a^{-1} \pmod{n} = x \pmod{n}$$